

CAPITULO ESTUDIANTEL ACM - ICPC UMSA

3

CAMPAMENTO DE PROGRAMACIÓN COMPETITIVA

ENERO 2019





3. Grafos I



Oscar Gauss Carvajal Yucra
Herbert Wilmer Quispe Churqui

Carrera de Informática
Facultad de Ciencias Puras y Naturales



Grafos I



- 1 Grafo
- 2 Clases de Grafos
 - Por su dirección
 - Multigrafo
 - Ponderación
 - Conectividad
- 3 Depth-First Search
- 4 Breadth First Search
- 5 Aplicaciones



1

Grafo



UMSA
la mejor

¿Qué es un grafo?



Un grafo $G = (V, E)$ es un conjunto no vacío de vértices o nodos $V = \{v_1, v_2, \dots, v_n\}$ y conjunto de aristas o arcos $E = \{e_1, e_2, \dots, e_m\}$, cada arista es un par de vértices $e = \{u, v\}$ no necesariamente distintos.

Vértices o Nodos

Aristas o Arcos



¿Qué es un grafo?



Un grafo $G = (V, E)$ es un conjunto no vacío de vértices o nodos $V = \{v_1, v_2, \dots, v_n\}$ y conjunto de aristas o arcos $E = \{e_1, e_2, \dots, e_m\}$, cada arista es un par de vértices $e = \{u, v\}$ no necesariamente distintos.

Vértices o Nodos

Aristas o Arcos



¿Qué es un grafo?



Un grafo $G = (V, E)$ es un conjunto no vacío de vértices o nodos $V = \{v_1, v_2, \dots, v_n\}$ y conjunto de aristas o arcos $E = \{e_1, e_2, \dots, e_m\}$, cada arista es un par de vértices $e = \{u, v\}$ no necesariamente distintos.

Vértices o Nodos



Aristas o Arcos



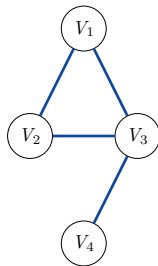
¿Qué es un grafo?



Un grafo $G = (V, E)$ es un conjunto no vacío de vértices o nodos $V = \{v_1, v_2, \dots, v_n\}$ y conjunto de aristas o arcos $E = \{e_1, e_2, \dots, e_m\}$, cada arista es un par de vértices $e = \{u, v\}$ no necesariamente distintos.

Vértices o Nodos

Aristas o Arcos



2

Clases de Grafos



UMSA
la mejor

Clases de Grafos



Los grafos se pueden clasificar por:

- Dirección
- Multigrafo
- Ponderación
- Conectividad





Los grafos se pueden clasificar por:

- Dirección
- Multigrafo
- Ponderación
- Conectividad





Los grafos se pueden clasificar por:

- Dirección
- Multigrafo
- Ponderación
- Conectividad



Por su dirección



Los grafos se clasifican en tres:

- Dirigido (Orientado)
- No dirigido
- Mixto



Por su dirección



Los grafos se clasifican en tres:

- Dirigido (Orientado)
- No dirigido
- Mixto



Por su dirección



Los grafos se clasifican en tres:

- Dirigido (Orientado)
- No dirigido
- Mixto



Por su dirección



Los grafos se clasifican en tres:

- Dirigido (Orientado)
- No dirigido
- Mixto



Por su dirección



Los grafos se clasifican en tres:

- Dirigido (Orientado)
- No dirigido
- Mixto



Multigrafo



Un multigrafo o pseudografo es un grafo que está facultado para tener autoejes o arcos paralelos.

Se llama **auto-eje** o bucle si el par de nodos es el mismo $e = \{u, u\}$.

Se llaman **aristas paralelas** o multi-arcos a las aristas con el mismo par de vértices se llaman.

Multidigrafo, es aquel multigrafo dirigido.





Un multigrafo o pseudografo es un grafo que está facultado para tener autoejes o arcos paralelos.

Se llama **auto-eje** o bucle si el par de nodos es el mismo $e = \{u, u\}$.

Se llaman **aristar paralelas** o multi-arcos a las aristas con el mismo par de vértices se llaman.

Multidigrafo, es aquel multigrafo dirigido.





Un multigrafo o pseudografo es un grafo que está facultado para tener autoejes o arcos paralelos.

Se llama **auto-eje** o bucle si el par de nodos es el mismo $e = \{u, u\}$.

Se llaman **aristas paralelas** o multi-arcos a las aristas con el mismo par de vértices se llaman.

Multidigrafo, es aquel multigrafo dirigido.



Ponderación



Los grafos se clasifican en dos:

- Grafos Ponderados
- Grafos No Ponderados



Ponderación



Los grafos se clasifican en dos:

- Grafos Ponderados
- Grafos No Ponderados





Los grafos se clasifican en dos:

- Grafos Ponderados
- Grafos No Ponderados





Los grafos se clasifican en dos:

- Grafos Ponderados
- Grafos No Ponderados



Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos
- Grafos Débilmente Conexos
- Grafos No Conexos





Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos
- Grafos Débilmente Conexos
- Grafos No Conexos





Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos

- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos

- Grafos Débilmente Conexos

- Grafos No Conexos





Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos
- Grafos Débilmente Conexos
- Grafos No Conexos





Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos
- Grafos Débilmente Conexos
- Grafos No Conexos





Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- **Grafos Fuertemente Conexos**
- Grafos Débilmente Conexos
- Grafos No Conexos





Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos
- Grafos Débilmente Conexos
- Grafos No Conexos



Para clasificar a los grafos por su conectividad necesitamos diferenciar a los grafos dirigidos y no dirigidos.

Los grafos no dirigidos por su conectividad se clasifican en:

- Grafos Conexos
- Grafos No Conexos

Los grafos dirigidos por su conectividad se clasifican en:

- Grafos Fuertemente Conexos
- Grafos Débilmente Conexos
- Grafos No Conexos



3

Depth-First Search



UMSA
la mejor

Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u y v , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
 - Cada vértice se visita una vez, y cada borde se recorre una vez
 - $O(n + m)$



Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



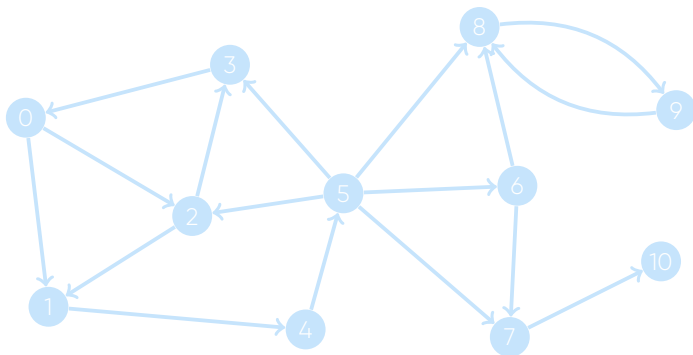
Depth-First Search



- Dado un grafo (ya sea dirigido o no dirigido) y dos vértices u , ¿Existe un camino de u a v ?
- Depth-first search es un algoritmo para encontrar dicha ruta, si existe o no.
- Atraviesa el grafo en orden por profundidad, a partir del vértice inicial u .
- En realidad, no tenemos que especificar una v (donde queremos llegar), ya que solo podemos dejar que visite todos los vértices accesibles desde u (y aún así tiene la misma complejidad de tiempo).
- Pero, ¿cuál es la complejidad del tiempo?
- Cada vértice se visita una vez, y cada borde se recorre una vez
- $O(n + m)$



Depth-First Search

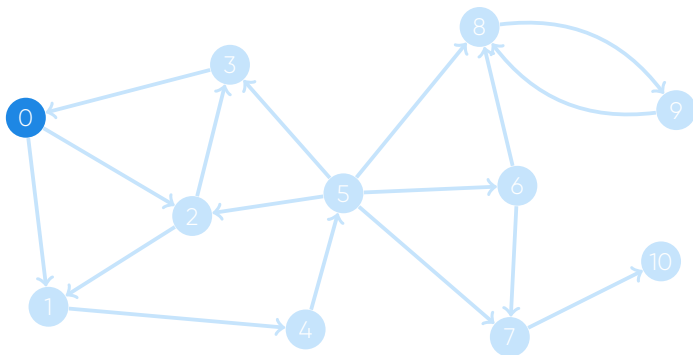


Pila: |

	0	1	2	3	4	5	6	7	8	9	10
visitado	0	0	0	0	0	0	0	0	0	0	0



Depth-First Search

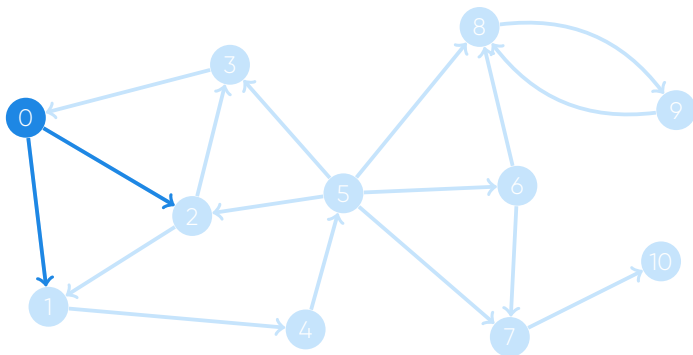


Pila: 0 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	0	0	0	0	0	0	0	0	0	0



Depth-First Search

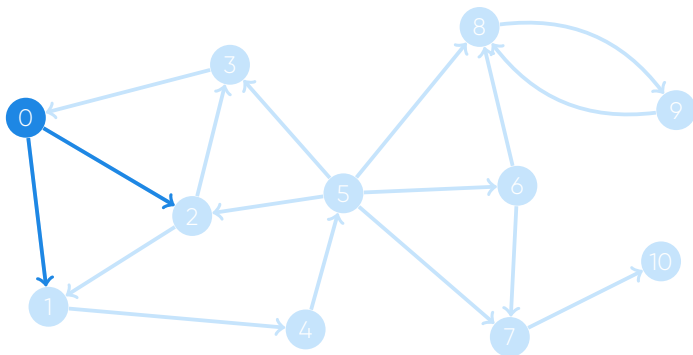


Pila: 0 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	0	0	0	0	0	0	0	0	0	0



Depth-First Search

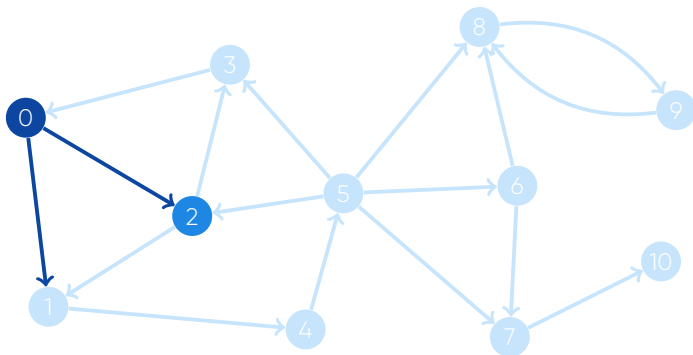


Pila: 0 | 2 1

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	0	0	0	0	0	0	0



Depth-First Search

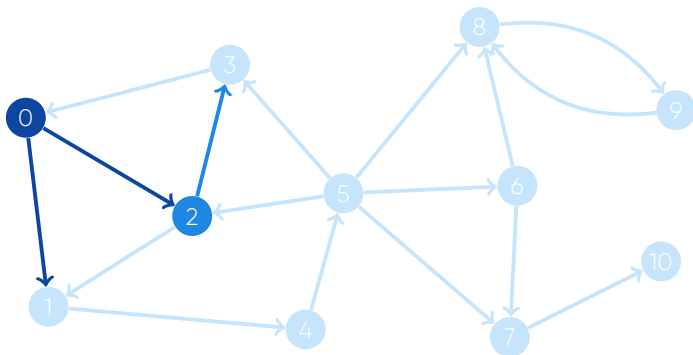


Pila: 2 | 1

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	0	0	0	0	0	0	0



Depth-First Search

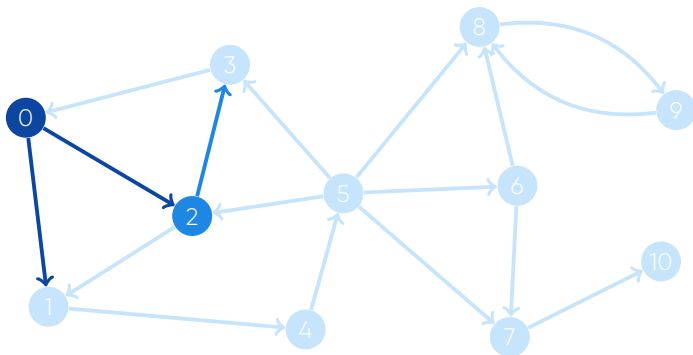


Pila: 2 | 1

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	0	0	0	0	0	0	0



Depth-First Search

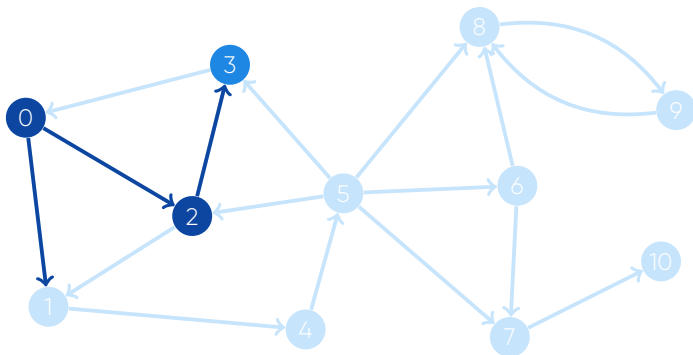


Pila: 2 | 3 1

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	0	0	0	0	0	0	0



Depth-First Search

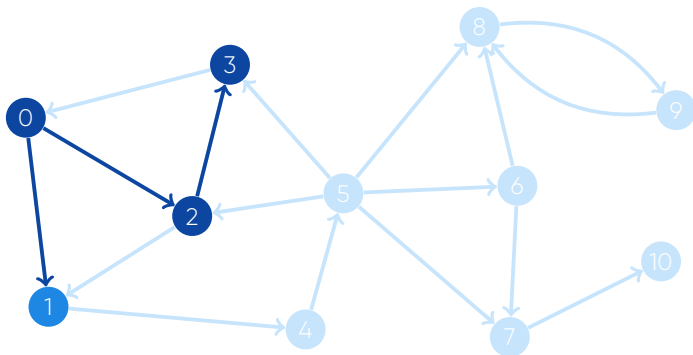


Pila: 3 | 1

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	0	0	0	0	0	0	0



Depth-First Search

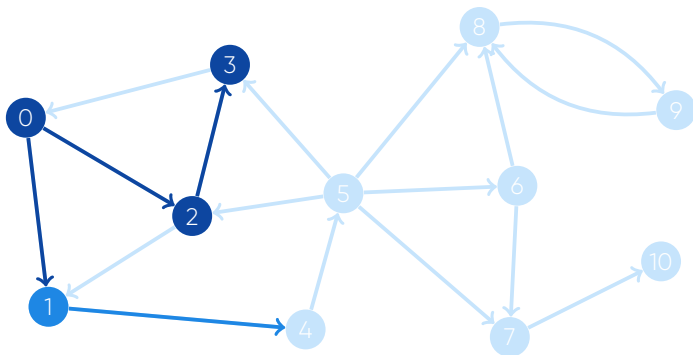


Pila: 1 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	0	0	0	0	0	0	0



Depth-First Search

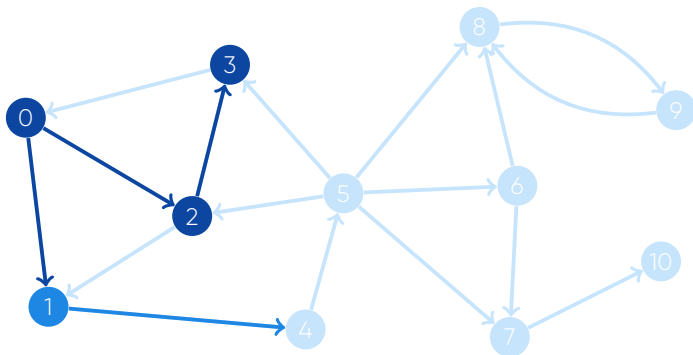


Pila: 1 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	0	0	0	0	0	0	0



Depth-First Search

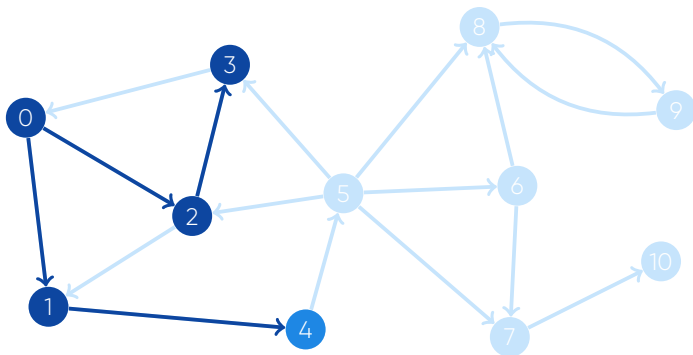


Pila: 1 | 4

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	0	0	0	0	0	0



Depth-First Search

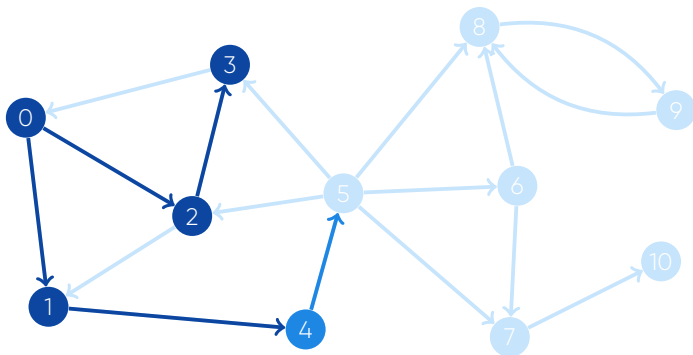


Pila: 4 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	0	0	0	0	0	0



Depth-First Search

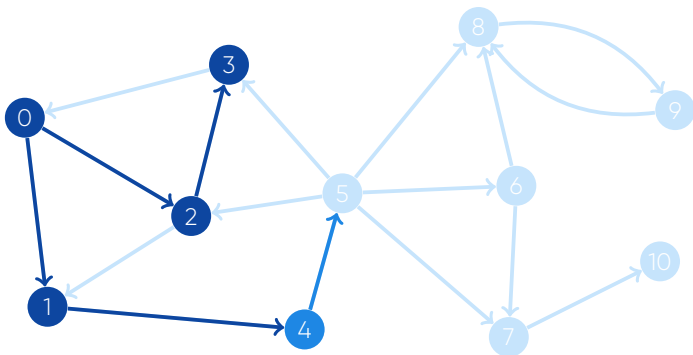


Pila: 4 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	0	0	0	0	0	0



Depth-First Search

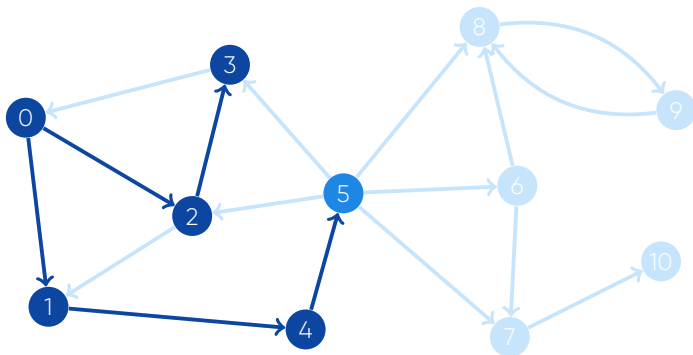


Pila: 4 | 5

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Depth-First Search

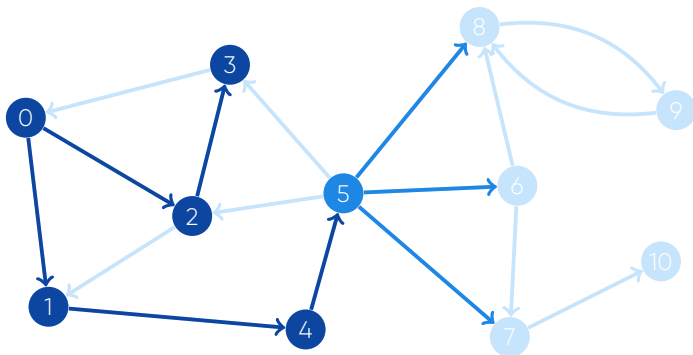


Pila: 5 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Depth-First Search

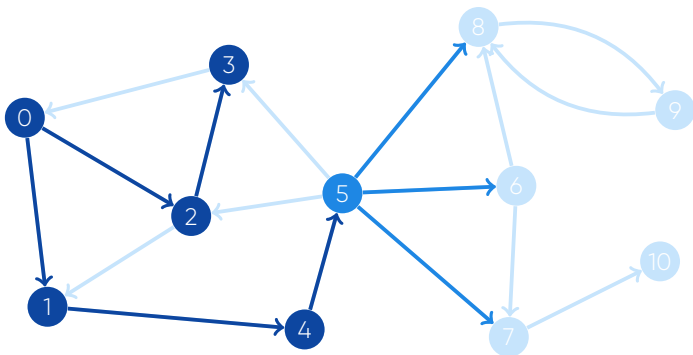


Pila: 5 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Depth-First Search

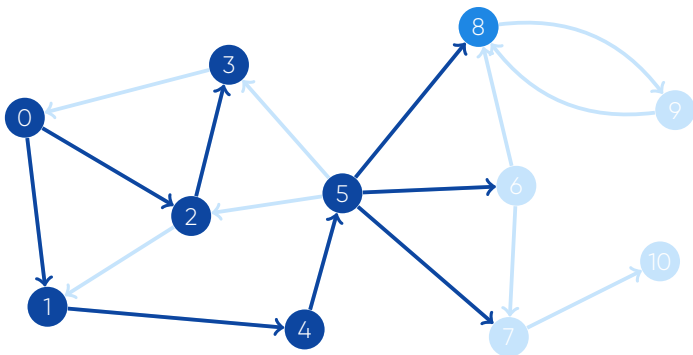


Pila: 5 | 8 6 7

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Depth-First Search

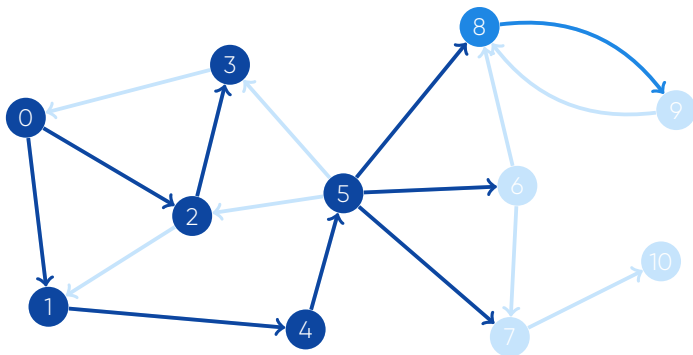


Pila: 8 | 6 7

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Depth-First Search

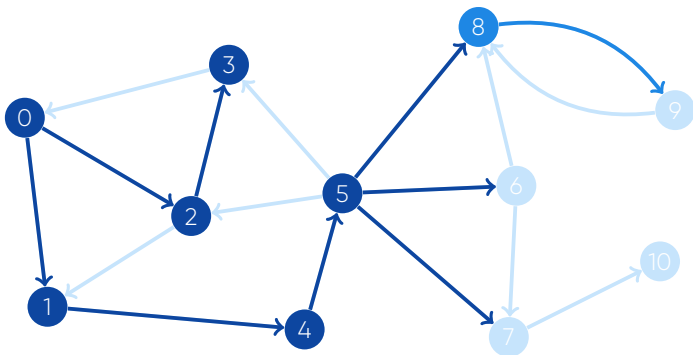


Pila: 8 | 6 7

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Depth-First Search

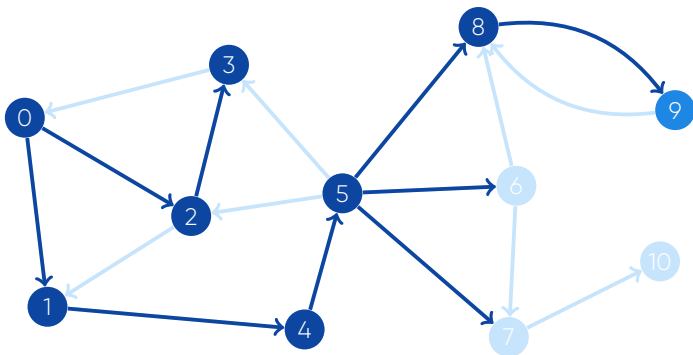


Pila: 8 | 9 6 7

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	0



Depth-First Search

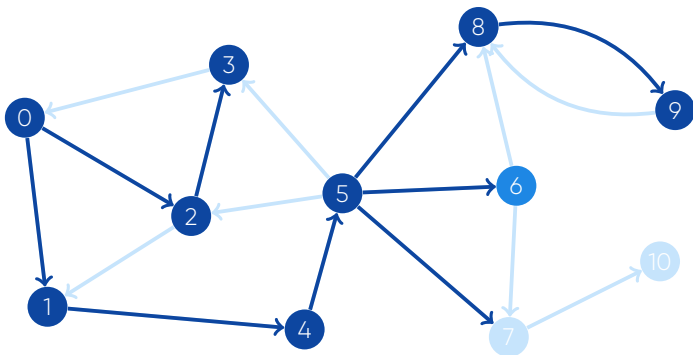


Pila: 9 | 6 7

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	0



Depth-First Search

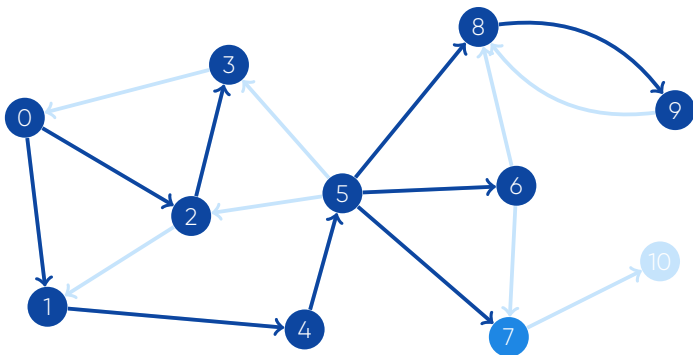


Pila: 6 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	0



Depth-First Search

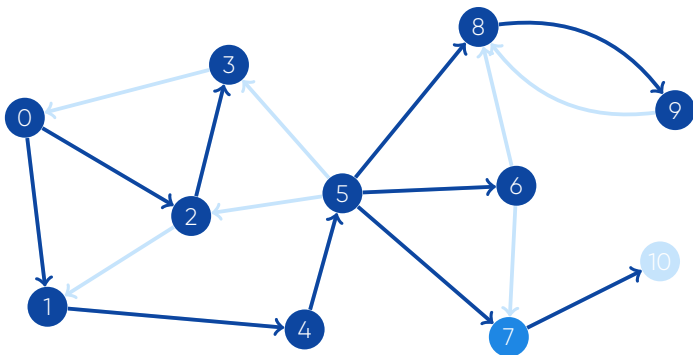


Pila: 7 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	0



Depth-First Search

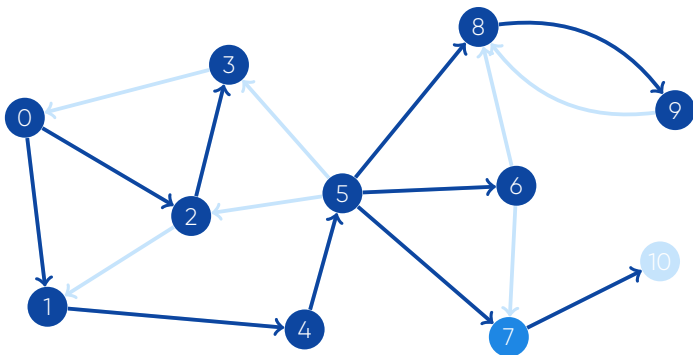


Pila: 7 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	0



Depth-First Search

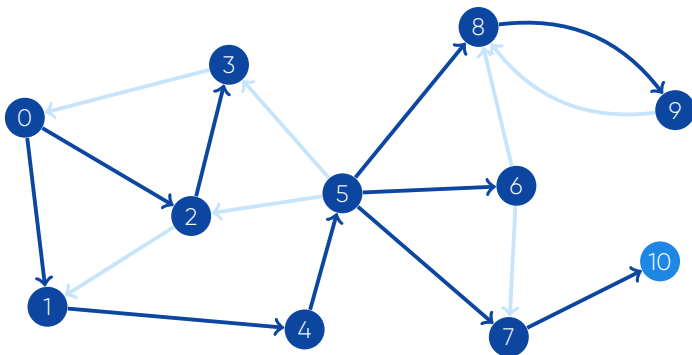


Pila: 7 | 10

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Depth-First Search

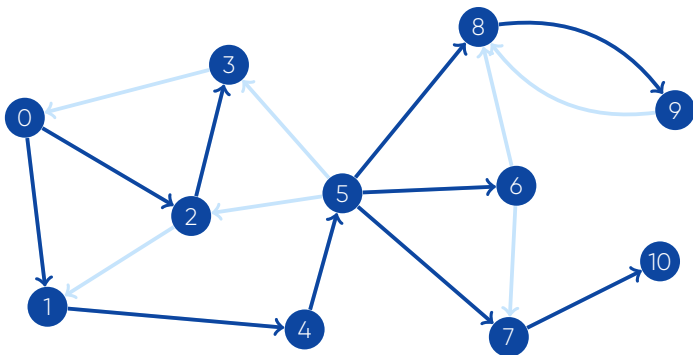


Pila: 10 |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Depth-First Search



Pila: |

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Depth-First Search



Codigo:

```
1 vector<int> G[1000];
2 vector<bool> visitado(1000, false);
3 void dfs(int u) {
4     if (visitado[u]) {
5         return;
6     }
7     visitado[u] = true;
8     for (int i = 0; i < G[u].size(); i++) {
9         int v = G[u][i];
10        dfs(v);
11    }
12 }
```



4

Breadth First Search



UMSA
la mejor

Breadth-first search



- Hay otro algoritmo de búsqueda llamado Breadth-first search
- La única diferencia es el orden en que visita los vértices
- Va en orden de distancia creciente desde el vértice fuente



Breadth-first search



- Hay otro algoritmo de búsqueda llamado Breadth-first search
- La única diferencia es el orden en que visita los vértices
- Va en orden de distancia creciente desde el vértice fuente



Breadth-first search



- Hay otro algoritmo de búsqueda llamado Breadth-first search
- La única diferencia es el orden en que visita los vértices
- Va en orden de distancia creciente desde el vértice fuente



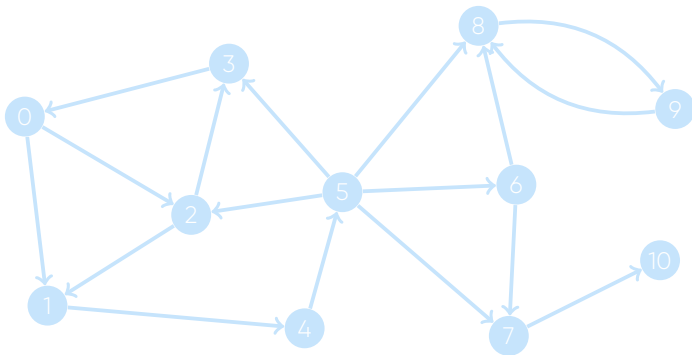
Breadth-first search



- Hay otro algoritmo de búsqueda llamado Breadth-first search
- La única diferencia es el orden en que visita los vértices
- Va en orden de distancia creciente desde el vértice fuente



Breadth-first search

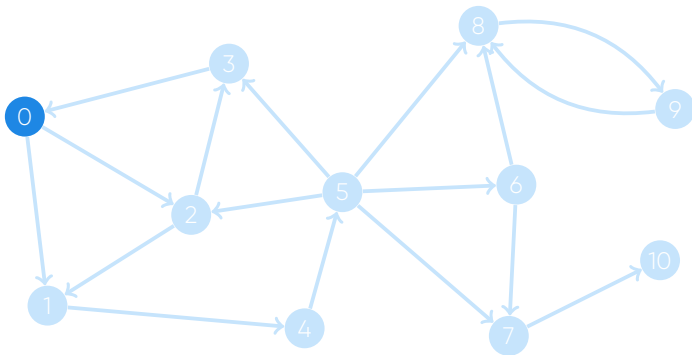


Cola:

	0	1	2	3	4	5	6	7	8	9	10
visitado	0	0	0	0	0	0	0	0	0	0	0



Breadth-first search

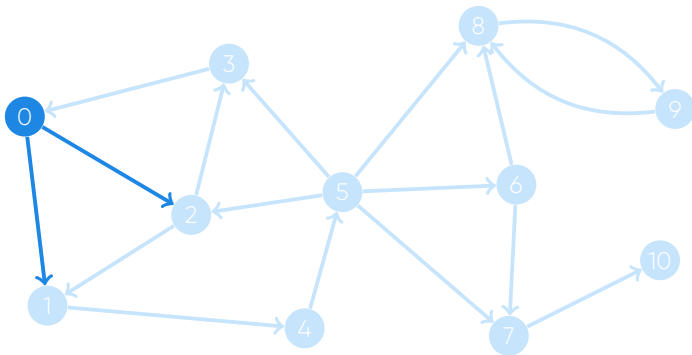


Cola: 0

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	0	0	0	0	0	0	0	0	0	0



Breadth-first search

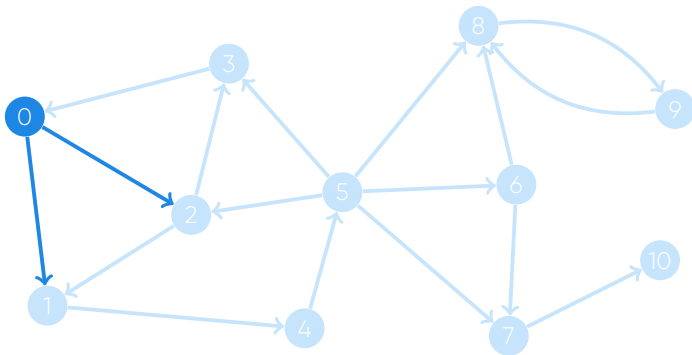


Cola: 0

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	0	0	0	0	0	0	0	0	0	0



Breadth-first search

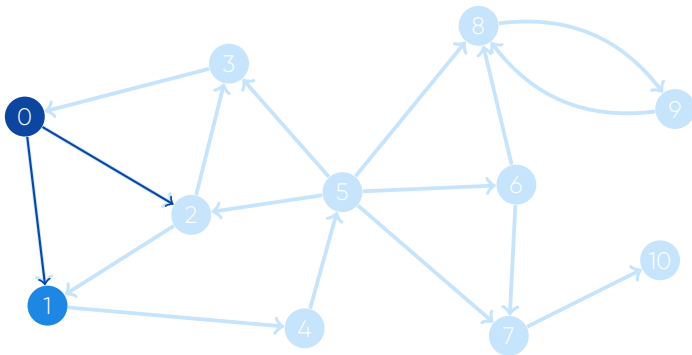


Cola: 0 1 2

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	0	0	0	0	0	0	0



Breadth-first search

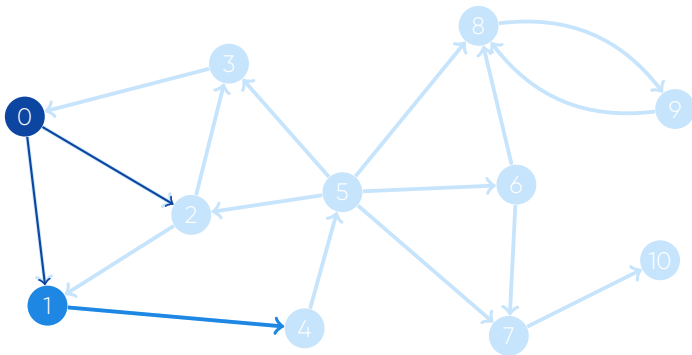


Cola: 1 2

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	0	0	0	0	0	0	0



Breadth-first search

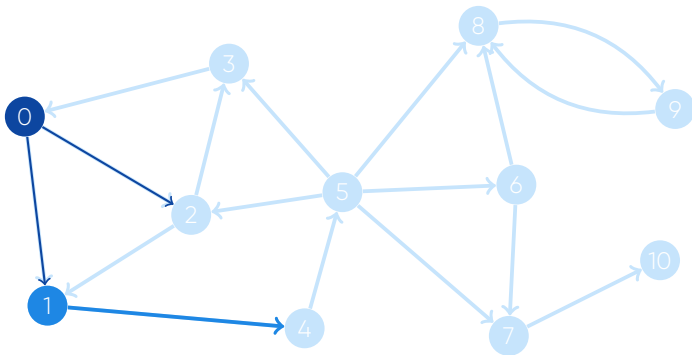


Cola: 1 2

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	0	0	0	0	0	0	0



Breadth-first search

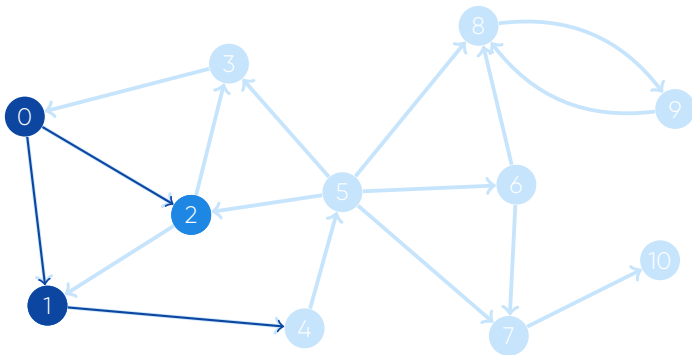


Cola: 1 2 4

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	1	0	0	0	0	0	0



Breadth-first search

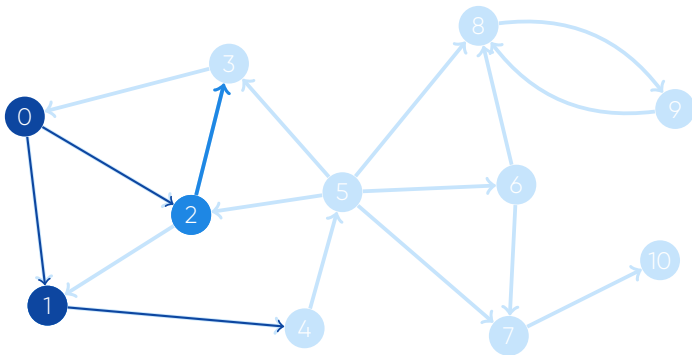


Cola: 2 4

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	1	0	0	0	0	0	0



Breadth-first search

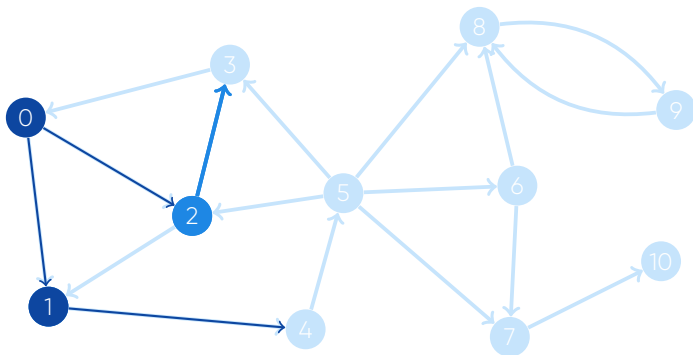


Cola: 2 4

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	0	1	0	0	0	0	0	0



Breadth-first search

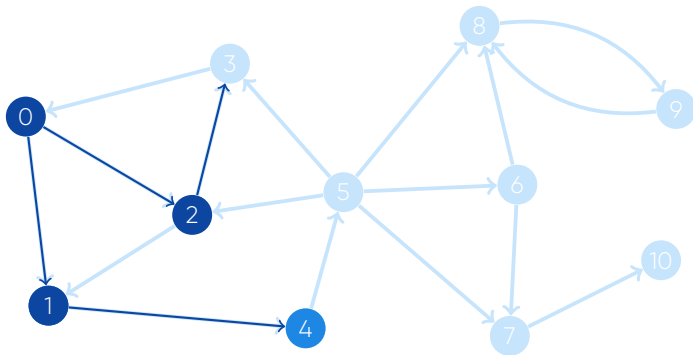


Cola: 2 4 3

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	0	0	0	0	0	0



Breadth-first search

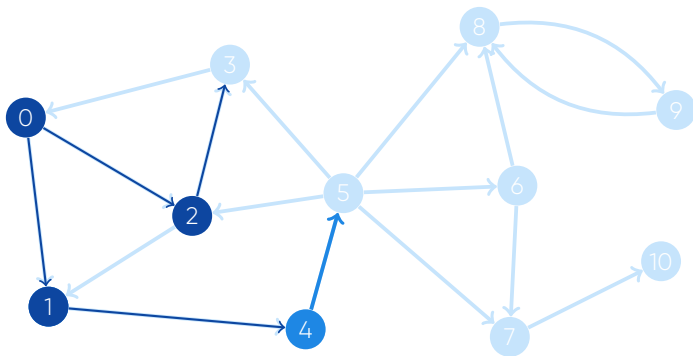


Cola: 4 3

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	0	0	0	0	0	0



Breadth-first search

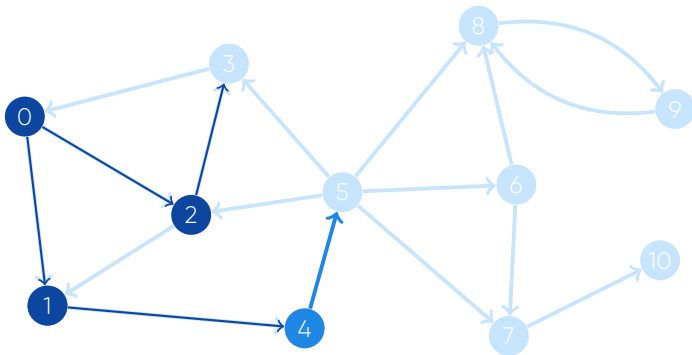


Cola: 4 3

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	0	0	0	0	0	0



Breadth-first search

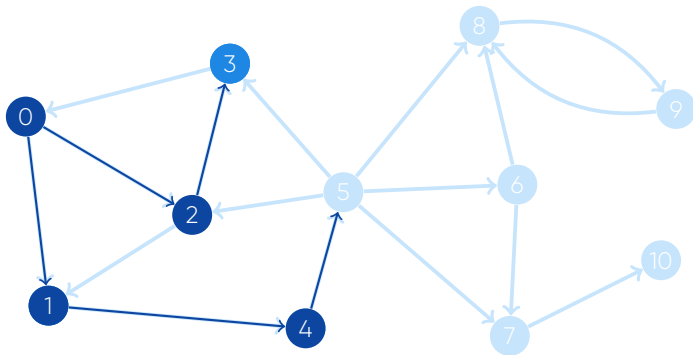


Cola: 4 3 5

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Breadth-first search

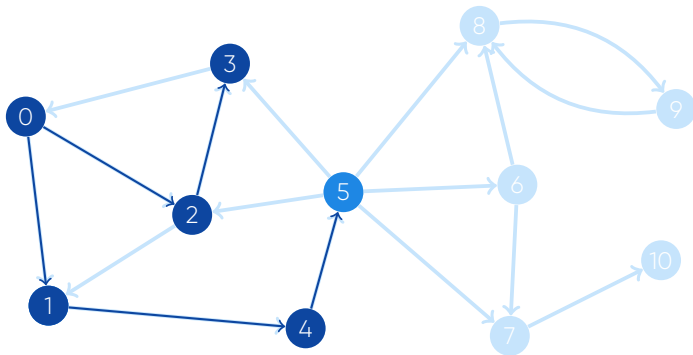


Cola: 3 5

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Breadth-first search

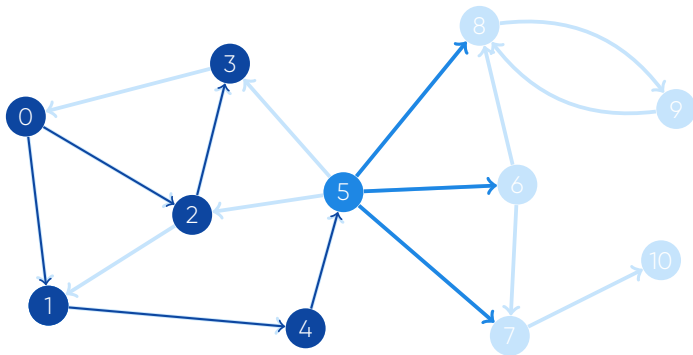


Cola: 5

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Breadth-first search

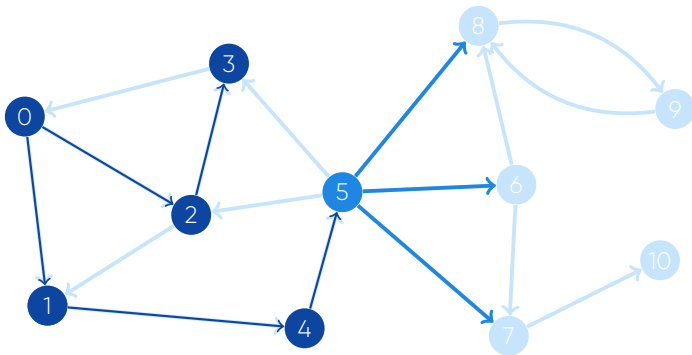


Cola: 5

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	0	0	0	0	0



Breadth-first search

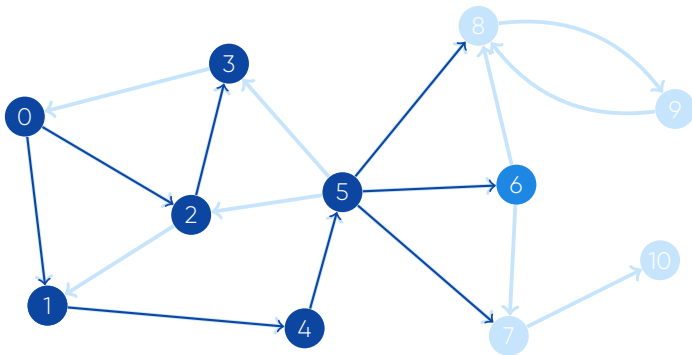


Cola: 5 6 7 8

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Breadth-first search

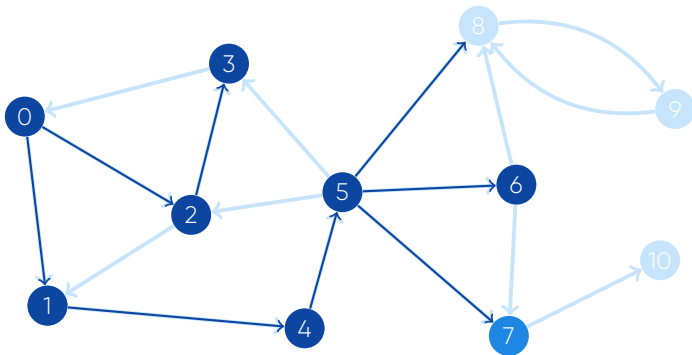


Cola: 6 7 8

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Breadth-first search

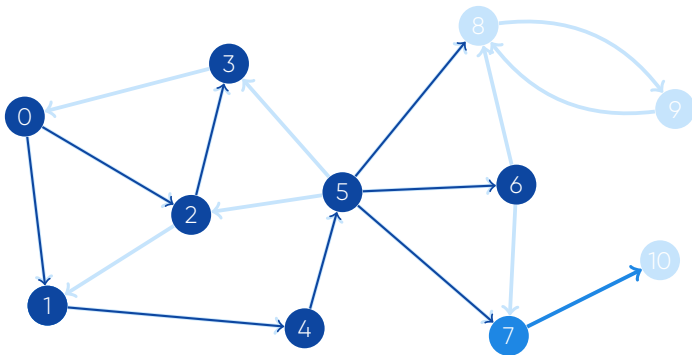


Cola: 7 8

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Breadth-first search

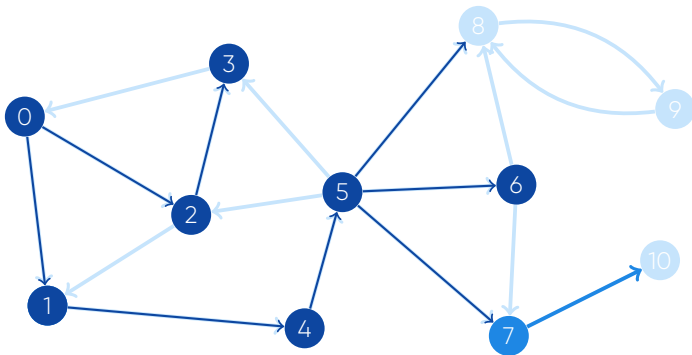


Cola: 7 8

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	0



Breadth-first search

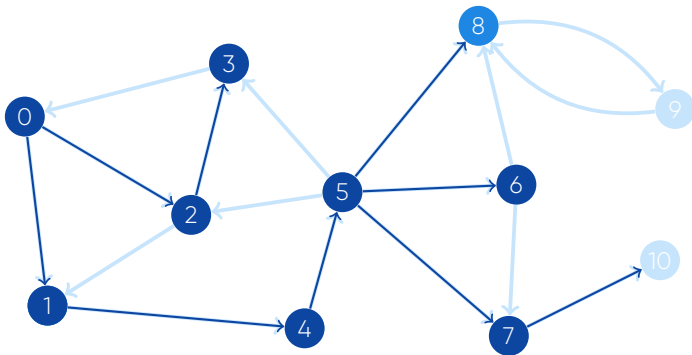


Cola: 7 8 10

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	1



Breadth-first search

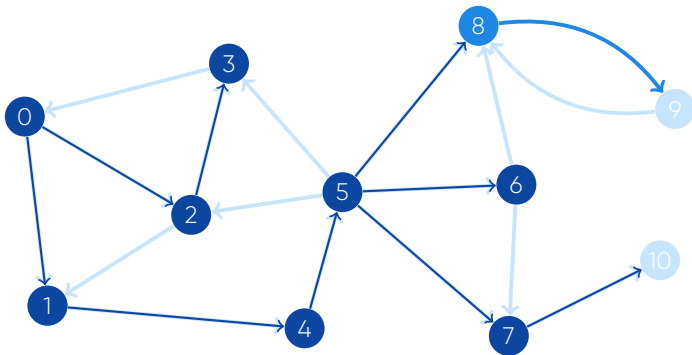


Cola: 8 10

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	1



Breadth-first search

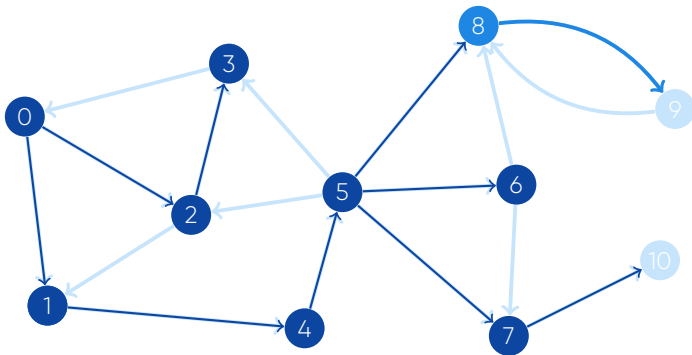


Cola: 8 10

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	0	1



Breadth-first search

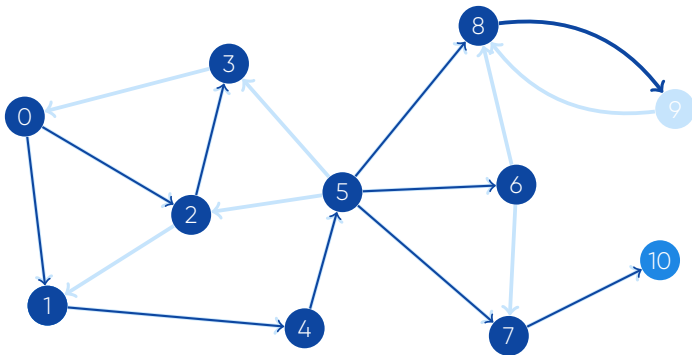


Cola: 8 10 9

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Breadth-first search

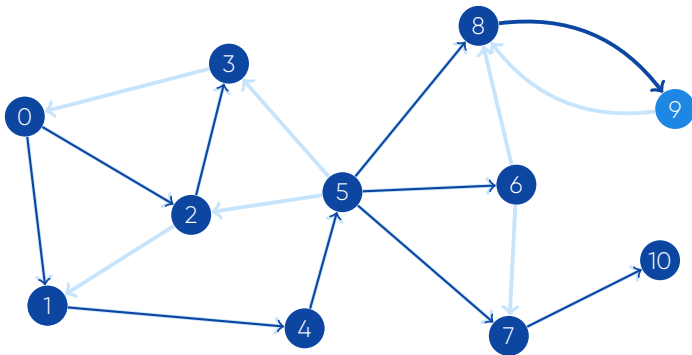


Cola: 10 9

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Breadth-first search

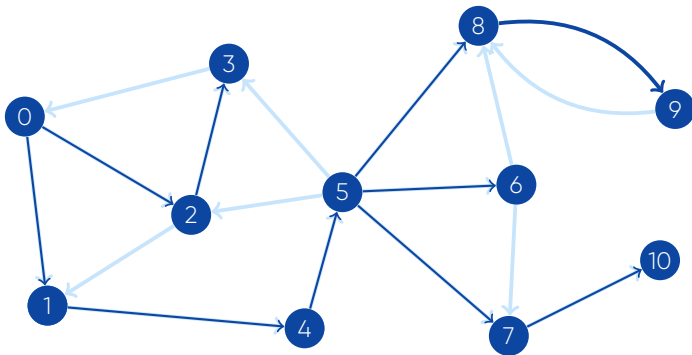


Cola: 9

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Breadth-first search



Cola:

	0	1	2	3	4	5	6	7	8	9	10
visitado	1	1	1	1	1	1	1	1	1	1	1



Breadth-first search



Codigo:

```
1  vector<int> G[1000];
2  vector<bool> visitado(1000, false);
3  queue<int> Q;
4  Q.push(start);
5  visitado[start] = true;
6  while (!Q.empty()) {
7      int u = Q.front(); Q.pop();
8      for (int i = 0; i < G[u].size(); i++) {
9          int v = G[u][i];
10         if (!visitado[v]) {
11             Q.push(v);
12             visitado[v] = true;
13         }
14     }
15 }
```



5

Aplicaciones



UMSA
la mejor

Ruta más corta en grafos no ponderados



- Tenemos un grafo sin ponderar, y queremos encontrar la ruta más corta de A a B
- Es decir, queremos encontrar una ruta desde A a B visitando el mínimo número de aristas.
- Simplemente se hace un BFS desde A , hasta que encontremos B
- Breadth-first search recorre los vértices en orden creciente de distancia desde el vértice de inicio.
- Si dejamos que la búsqueda continúe en todo el grafo, tendremos las rutas más cortas del vertice A a todos los demás vértices.
- Ruta más corta de A a todos los demás vértices: $O(n + m)$



Ruta más corta en grafos no ponderados



- Tenemos un grafo sin ponderar, y queremos encontrar la ruta más corta de A a B
- Es decir, queremos encontrar una ruta desde A a B visitando el mínimo número de aristas.
- Simplemente se hace un BFS desde A , hasta que encontremos B
- Breadth-first search recorre los vértices en orden creciente de distancia desde el vértice de inicio.
- Si dejamos que la búsqueda continúe en todo el grafo, tendremos las rutas más cortas del vertice A a todos los demás vértices.
- Ruta más corta de A a todos los demás vértices: $O(n + m)$



Ruta más corta en grafos no ponderados



- Tenemos un grafo sin ponderar, y queremos encontrar la ruta más corta de A a B
- Es decir, queremos encontrar una ruta desde A a B visitando el mínimo número de aristas.
- Simplemente se hace un BFS desde A , hasta que encontremos B
- Breadth-first search recorre los vértices en orden creciente de distancia desde el vértice de inicio.
- Si dejamos que la búsqueda continúe en todo el grafo, tendremos las rutas más cortas del vertice A a todos los demás vértices.
- Ruta más corta de A a todos los demás vértices: $O(n + m)$



Ruta más corta en grafos no ponderados



- Tenemos un grafo sin ponderar, y queremos encontrar la ruta más corta de A a B
- Es decir, queremos encontrar una ruta desde A a B visitando el mínimo número de aristas.
- Simplemente se hace un BFS desde A , hasta que encontremos B
- Breadth-first search recorre los vértices en orden creciente de distancia desde el vértice de inicio.
- Si dejamos que la búsqueda continúe en todo el grafo, tendremos las rutas más cortas del vertice A a todos los demás vértices.
- Ruta más corta de A a todos los demás vértices: $O(n + m)$



Ruta más corta en grafos no ponderados



- Tenemos un grafo sin ponderar, y queremos encontrar la ruta más corta de A a B
- Es decir, queremos encontrar una ruta desde A a B visitando el mínimo número de aristas.
- Simplemente se hace un BFS desde A , hasta que encontremos B
- Breadth-first search recorre los vértices en orden creciente de distancia desde el vértice de inicio.
- Si dejamos que la búsqueda continúe en todo el grafo, tendremos las rutas más cortas del vertice A a todos los demás vértices.
- Ruta más corta de A a todos los demás vértices: $O(n + m)$



Ruta más corta en grafos no ponderados



- Tenemos un grafo sin ponderar, y queremos encontrar la ruta más corta de A a B
- Es decir, queremos encontrar una ruta desde A a B visitando el mínimo número de aristas.
- Simplemente se hace un BFS desde A , hasta que encontremos B
- Breadth-first search recorre los vértices en orden creciente de distancia desde el vértice de inicio.
- Si dejamos que la búsqueda continúe en todo el grafo, tendremos las rutas más cortas del vertice A a todos los demás vértices.
- Ruta más corta de A a todos los demás vértices: $O(n + m)$



Ruta más corta en grafos no ponderados



Codigo:

```
1 vector<int> G[1000];
2 vector<bool> dist(1000, -1);
3 queue<int> Q;
4 Q.push(A);
5 dist[A] = 0;
6 while (!Q.empty()) {
7     int u = Q.front(); Q.pop();
8     for (int i = 0; i < G[u].size(); i++) {
9         int v = G[u][i];
10        if (dist[v] == -1) {
11            Q.push(v);
12            dist[v] = 1 + dist[u];
13        }
14    }
15 }
16 printf("%d\\n", dist[B]);
```



Componentes conectados



- Etiquetar a los componentes conexos en grafos no dirigidos.
- Simplemente se hace un DFS desde cualquier nodo, todos los nodos que toque el DFS están en una componente conexa, seguimos, lanzamos otro DFS de otro nodo no visitado y todos los nodos que toque ese DFS están en otra componente conexa, y así hasta visitar todos los nodos.
- El tiempo de complejidad es $O(n + m)$
- Otra manera es aplicando Union - Find



Componentes conectados



- Etiquetar a los componentes conexos en grafos no dirigidos.
- Simplemente se hace un DFS desde cualquier nodo, todos los nodos que toque el DFS están en una componente conexa, seguimos, lanzamos otro DFS de otro nodo no visitado y todos los nodos que toque ese DFS están en otra componente conexa, y así hasta visitar todos los nodos.
- El tiempo de complejidad es $O(n + m)$
- Otra manera es aplicando Union - Find



Componentes conectados



- Etiquetar a los componentes conexos en grafos no dirigidos.
- Simplemente se hace un DFS desde cualquier nodo, todos los nodos que toque el DFS están en una componente conexa, seguimos, lanzamos otro DFS de otro nodo no visitado y todos los nodos que toque ese DFS están en otra componente conexa, y así hasta visitar todos los nodos.
- El tiempo de complejidad es $O(n + m)$
- Otra manera es aplicando Union - Find



Componentes conectados



- Etiquetar a los componentes conexos en grafos no dirigidos.
- Simplemente se hace un DFS desde cualquier nodo, todos los nodos que toque el DFS están en una componente conexa, seguimos, lanzamos otro DFS de otro nodo no visitado y todos los nodos que toque ese DFS están en otra componente conexa, y así hasta visitar todos los nodos.
- El tiempo de complejidad es $O(n + m)$
- Otra manera es aplicando Union - Find



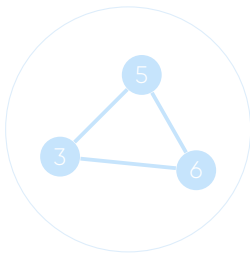
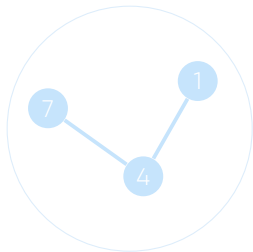
Componentes conectados



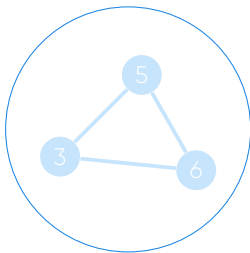
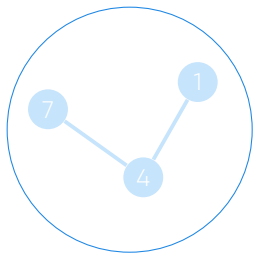
- Etiquetar a los componentes conexos en grafos no dirigidos.
- Simplemente se hace un DFS desde cualquier nodo, todos los nodos que toque el DFS están en una componente conexa, seguimos, lanzamos otro DFS de otro nodo no visitado y todos los nodos que toque ese DFS están en otra componente conexa, y así hasta visitar todos los nodos.
- El tiempo de complejidad es $O(n + m)$
- Otra manera es aplicando Union - Find



Componentes conectados



Componentes conectados

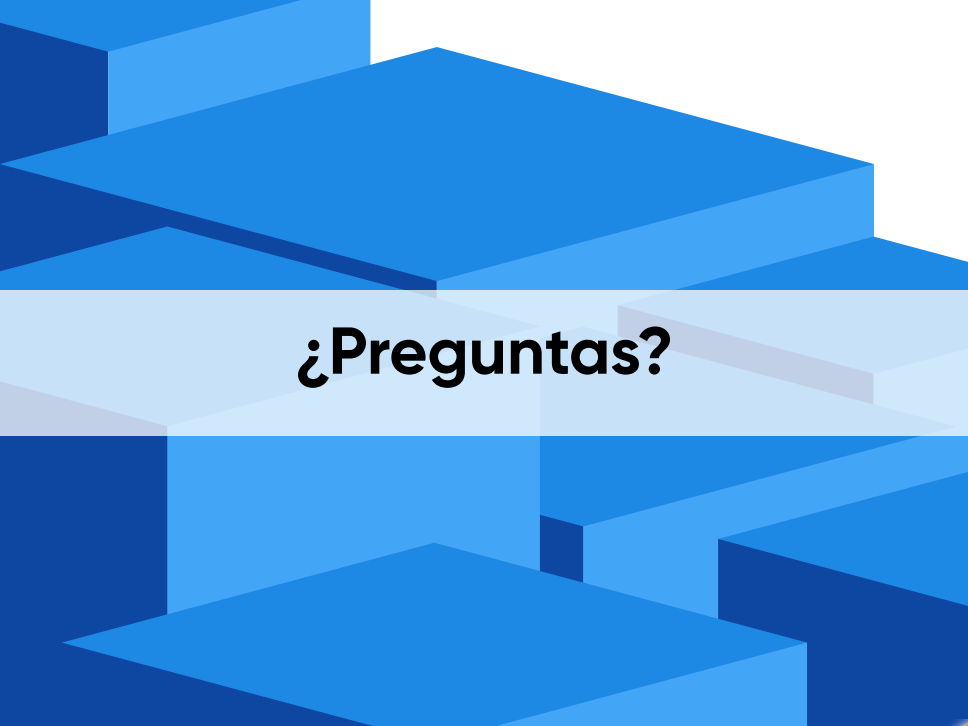


Componentes conectados



```
1 vector<int> G[1000];
2 vector<int> componente(1000, -1);
3 void buscar_componente(int cur_comp, int u) {
4     if (componente[u] != -1) {
5         return;
6     }
7     componente[u] = cur_comp;
8     for (int i = 0; i < G[u].size(); i++) {
9         int v = G[u][i];
10        find_component(cur_comp, v);
11    }
12 }
13 int componentes = 0;
14 for (int u = 0; u < n; u++) {
15     if (componente[u] == -1) {
16         buscar_componente(componentes, u);
17         componentes++;
18     }
19 }
```



The background is an abstract composition of various blue geometric shapes, including squares and rectangles, arranged in a way that creates a sense of depth and perspective. The colors range from a deep navy blue to a bright, light blue. A prominent white horizontal band runs across the center of the image, serving as a backdrop for the text.

¿Preguntas?

The background is an abstract composition of various geometric shapes, primarily squares and rectangles, in different shades of blue (dark blue, medium blue, light blue) and white. These shapes are arranged in a way that creates a sense of depth and perspective, resembling a stylized architectural structure or a series of steps. The lighting appears to come from the upper left, casting shadows that enhance the three-dimensional effect.

Gracias!